

SUNO LLM WORKSHOP — MASTER ACTION PLAN PROJECT ID: SUNOWORKSHOP-2026-08-A CURRENT
TARGET: Suno v5.5 METHOD: build → test → listen → revise theory → build again

MISSION

Develop a practical LLM-assisted workshop for inventing songs, constructing their internal systems, translating those systems into Suno prompts, learning from generations, and preserving useful discoveries without reducing the process to conventional prompt optimization.

The workshop must support:

- genre and sonic invention
- song architecture
- lyric and narrative architecture
- hooks and listener attention
- experimental pseudo-code prompting
- Styles/Lyrics/Exclude compilation
- prompt debugging
- generation comparison
- model-version tracking
- preservation of productive friction, ambiguity, and emergent results

CORE RULE

Do not ask Suno to execute an abstract specification that has nothing musical to act upon.

Every important structural relationship should connect:

musical object → musical behavior → structural or lyrical location → persistence, mutation, transfer, or termination

WORKING MODEL

Styles = global physics and interpretive engine

Lyrics plain text = vocal-eligible temporal material

Lyrics brackets = local conceptual, structural, performance, or production influence

Exact lyric phrases = possible structural anchors and trigger points

Exclude = weak boundary against specific unwanted resolutions

Settings = part of the prompt experiment, not an afterthought

Suno output = negotiated musical realization, not deterministic execution

===== PHASE 0 — ESTABLISH THE WORKSHOP
=====

PURPOSE

Create the minimum organizational skeleton needed to work without losing prompts, outputs, findings, and contradictions.

0.1 Create the canonical project map

Create:

00_Project_Map 01_Action_Plan 02_Working_Theory 03_Taste_Profile 04_Prompt_Grammar
05_Song_Architecture_Workbook 06_Experiment_Ledger 07_Model_Behavior_Registry
08_Prompt_and_Song_Archive 09_Open_Questions 99_Archive

RULE

Live documents remain current and compact.

Superseded material moves intact to the archive rather than disappearing or producing Final_Final_v7 files.

0.2 Establish confidence labels

Every model-behavior statement receives one label:

OBSERVATION Something happened in one generation or generation pair.

REPEATED OBSERVATION It happened several times under similar conditions.

WORKING HYPOTHESIS A plausible explanation that guides experiments.

PROVISIONAL RULE It survives several prompt families or controlled comparisons.

VERSION-BOUND RULE It appears reliable for one Suno version or settings regime.

OPEN CONTRADICTION Good evidence points in more than one direction.

0.3 Establish model metadata

Every experiment records:

- date

- Suno model
- prompt mode
- Styles
- Lyrics
- Exclude
- Weirdness
- Style Influence
- relevant Persona, Voice, Custom Model, or audio input
- generation count
- which output is being discussed

WRAP CONDITION

We can retrieve any experiment and know exactly what was submitted and under what conditions.

===== PHASE 1 — INGEST AND MAP THE EXISTING
CORPUS =====

PURPOSE

Extract the real methods already developed across GenreLab, Hook Ideas, Model Insights, SUNO BEST, and the current discussion.

Do not summarize everything chronologically. Reorganize it by function.

1.1 Extract the creative method

Compile:

- source-pool construction
- genre collection
- instrument and vocal vocabularies
- portmanteau and compound-genre methods
- friction-based recombination
- impossible acoustic objects
- recursive recombination strategies
- methods that generated unusually strong songs
- approaches that repeatedly became conventional

1.2 Extract the communication hypotheses

Organize findings under:

- Styles behavior
- Lyrics behavior
- bracket behavior
- plain-text vocalization

- exact phrase anchoring
- local versus distant definition
- repetition and persistence
- stop, clear, swap, toggle, reset, and return
- code-shaped material
- roleplay and performance frameworks
- character limits
- Exclude behavior
- settings effects
- pronunciation and phonetic prompting

1.3 Extract the song-architecture methods

Compile:

- invariant
- engine
- counterforce
- mutation path
- attention path
- reveal and payoff
- hook chaining
- breathers
- callback systems
- parallel structures
- multiple levels of interpretation
- first-listen versus repeat-listen behavior
- endings that recontextualize the opening

1.4 Extract Nathan's taste profile

Do not reduce this to favorite genres.

Record:

- preferred kinds of strangeness
- preferred vocal restraint
- desired looseness and tightness
- acceptable lightness
- unwanted cabaret and revue behavior
- productive rhythmic mismatch
- useful miscalculation
- acoustic versus electronic preferences
- melodic "bippiness"
- semantic and phonetic nonsense
- tolerance for conventional coherence
- signs that an optimization has sterilized the material

QUICK EXPERIMENT 1 — CORPUS CHECK

Select three archived prompts:

A. one exceptionally successful strange prompt B. one coherent but more conventional prompt C. one interesting partial failure

For each, identify:

- invariant
- global engine
- local control layer
- unresolved conflicts
- likely sources of conventionality
- emergent features worth retaining

Do not regenerate yet.

WRAP CONDITION

The workshop can describe what Nathan actually does without forcing it into ordinary songwriting or prompt-engineering assumptions.

===== PHASE 2 — BUILD THE WORKING THEORY
=====

PURPOSE

Construct a theory useful enough to guide prompts but modest enough to remain revisable.

2.1 Separate three different systems

CREATIVE INVENTION How ideas, genres, sounds, and structures are generated.

SONG ARCHITECTURE How musical and lyrical elements interact over time.

SUNO COMMUNICATION How the song model is represented in Styles, Lyrics, Exclude, settings, and source audio.

Never mistake a strong theory in one layer for a rule in another.

2.2 Define the conceptual structurizer model

The prompt does not primarily function as a program.

It presents:

- musical entities
- identities
- affinities
- oppositions
- containment
- sequence
- transformation
- recurrence
- persistence
- interruption
- return

Syntax helps establish relationships but has little value without musically meaningful terms.

2.3 Define semantic distance

Create three scope classes:

GLOBAL Must shape most or all of the song; belongs mainly in Styles.

REGIONAL Must govern a section, phase, voice, motif family, or active state.

LOCAL Must happen near a phrase, lyric, cue, instrument, or transition.

Working principle:

The more local the desired result, the nearer its meaningful description should be to the material it acts upon.

2.4 Define identity and conservation

For every important musical object, determine what must survive transformation:

- contour
- rhythm
- phrase
- phonemes
- instrument identity
- voice identity
- harmonic role
- dramatic function
- recording texture

This prevents “transformation” from simply meaning replacement.

2.5 Define productive conflict

Classify every contradiction as:

- counterforce
- ambiguity
- alternating jurisdiction
- unstable compromise
- deliberate contamination
- broken wiring
- dead remnant

Only the last two are automatically repaired.

QUICK EXPERIMENT 2 — LOCAL BINDING

Use one short lyric and one simple musical engine.

Generate matched pairs:

A. distant declaration: Define HOOK and FRICTION early, invoke labels later.

B. local definition: Restate the musical consequence where HOOK and FRICTION act.

Keep all other fields and settings unchanged.

Listen for:

- hook identity
- timing of friction
- locality
- persistence
- overall conventionality
- whether local reminder consumes too much prompt attention

WRAP CONDITION

The theory explains observed behavior while clearly distinguishing observations, hypotheses, and provisional rules.

===== PHASE 3 — BUILD THE SONG ARCHITECTURE
WORKBENCH =====

PURPOSE

Prevent the LLM from jumping directly from idea to prompt.

3.1 Create the Song Seed

Record:

- central attraction
- emotional or conceptual problem
- primary sound-world
- first strong image, phrase, groove, or hook
- what the song must not become

3.2 Create the Song Machine

Define:

INVARIANT What remains recognizable.

ENGINE What continuously generates motion.

COUNTERFORCE What resists or distorts the engine.

MUTATION What changes each cycle.

MEMORY What returns carrying evidence of prior events.

FAILURE MODE What happens when the system cannot reconcile itself.

RETURN What comes back, and how it has changed.

3.3 Create the Attention Architecture

Map:

- first anomaly
- first payoff
- first proof of musical competence
- second promise
- breather
- escalation
- concealed callback
- convergence
- final fulfillment
- replay reward

The intended path may resemble:

confusion → recognition → pleasure → renewed uncertainty → anticipation → recontextualization

rather than merely:

intro → verse → chorus

3.4 Create the Depth Architecture

Test at least three readings:

SURFACE Does it work immediately?

REPEAT Do patterns, jokes, transformations, or callbacks emerge?

DEEP Does the work support interpretation beyond deciphering a trick?

3.5 Create the Friction Map

For each pair of important elements, record whether they:

- reinforce
- oppose
- stagger
- imitate
- infect
- interrupt
- exchange roles
- remain incompatible
- reconcile temporarily

QUICK EXPERIMENT 3 — COHERENCE WITHOUT CONVENTIONALITY

Take one stable song machine and make three Styles engines:

A. fully reconciled B. two clearly competing jurisdictions C. same as B, but one invariant is strongly protected

Use the same Lyrics.

Determine whether protected invariance allows greater surrounding craziness without losing the song.

WRAP CONDITION

A complete conceptual song exists before prompt compilation begins.

===== PHASE 4 — BUILD THE PROMPT COMPILER
=====

PURPOSE

Translate a song architecture into character-efficient Suno inputs.

4.1 Compile Styles

Styles should prioritize:

- interpretive frame
- principal performers or sound sources
- global groove and meter
- invariant
- engine
- key counterforce
- trigger grammar
- persistence behavior
- broad transformation path
- recording ontology
- targeted anti-priors

Avoid filling Styles with decorative adjectives that do not participate in the system.

4.2 Compile Lyrics

Lyrics contain:

- plain vocal material
- line and stanza geometry
- exact trigger phrases
- one-line local bracket directions
- section and phase markers
- locally repeated identity reminders
- explicit stop, reset, clear, swap, or return points where needed

Do not rely on:

- indentation
- unclosed multiline bracket scope
- distant symbolic lookup
- undefined operators
- punctuation alone to carry musical meaning

4.3 Compile Exclude

Use Exclude to resist specific likely defaults, not to describe the desired song negatively.

Every important exclusion should have a positive replacement elsewhere.

Example:

Exclude: Broadway cadence

Positive replacement: tight underplayed close-miked delivery; clipped phrase endings; no audience-facing projection

4.4 Perform a wiring audit

Check:

- every trigger phrase matches exactly
- every stop has a start
- every local transformation names or clearly touches an active object
- every recurring label carries enough local musical reminder
- no plain text remains unless vocalization is acceptable
- no bracketed lyric is essential sung content
- no old cue survives without a job
- transformations preserve the intended invariant

4.5 Perform a character audit

For every grouping character, label, and repeated phrase, ask:

Does this clarify a relationship? Does this reinforce identity? Does this provide musical information? Does this create a useful lyric or visual boundary?

If not, remove it.

QUICK EXPERIMENT 4 — COMPRESSION

Create:

A. fully explicit prompt B. compact prompt with grouped global states and local reminders C. overcompressed prompt

Compare:

- fidelity
- musical quality
- distinctiveness
- structural coherence
- prompt-character efficiency
- unexpected inventions

WRAP CONDITION

The compiler produces prompts that remain musically legible after character reduction.

===== PHASE 5 — BUILD THE EXPERIMENT BENCH
=====

PURPOSE

Learn quickly without converting every song into a sterile laboratory test.

5.1 Use paired tests

Change one major variable per comparison whenever the purpose is model learning.

5.2 Use exploratory tests

When the purpose is creative discovery, allow several interacting changes—but label the result exploratory rather than causal evidence.

5.3 Standard experiment families

BRACKET TEST Plain text versus bracketed text.

LOCALITY TEST Distant definition versus definition at point of use.

TRIGGER TEST Styles declaration alone versus Styles plus exact Lyrics anchor.

PERSISTENCE TEST No stop versus explicit stop versus explicit reset.

GROUPING TEST One grouped bracket versus separate bracketed lines.

CODE TEST Real code, sound-substituted code, and conceptual pseudo-code.

ROLEPLAY TEST Literal command versus a performer or machine embodying the behavior.

SETTINGS TEST Fixed prompt across selected Weirdness and Style Influence values.

INVARIANT TEST Weak, moderate, and heavily reinforced identity.

FRICTION TEST Resolved hybrid versus incompatible coexistence.

VOCAL-TIGHTNESS TEST Punctuation, range, projection, phrase length, and delivery terms.

5.4 Standard listening audit

For every result:

ACTUAL VOCALS What was sung, spoken, skipped, repeated, or converted into refrain?

ACTUAL STRUCTURE What sections or transformations occurred?

ACTUAL SONIC OBJECTS Which requested or invented sounds appeared?

ACTUAL RELATIONSHIPS Which elements governed, followed, interrupted, or merged?

ACTUAL PERSISTENCE Which events remained active after their apparent trigger?

CONVENTIONALITY Where did Suno resolve uncertainty through familiar song behavior?

EMERGENCE What valuable feature appeared that the prompt did not explicitly specify?

5.5 Promotion rule

Never convert one good song directly into a universal prompting rule.

Replicate the behavior in another prompt family first.

WRAP CONDITION

The workshop can distinguish a creative success from a reliable control finding.

===== PHASE 6 — BUILD THE GENRELAB AND RECOMBINATION ENGINE =====

PURPOSE

Formalize invention without turning it into random genre agglutination.

6.1 Maintain real source pools

Compile separately:

- genre traditions
- formal structures
- rhythms and meters
- vocal practices
- instrumental techniques
- acoustic materials
- performance situations
- recording technologies
- narrative forms
- production failures
- physical sound mechanisms

6.2 Recombine by relationship, not merely addition

Methods include:

DOMINANCE One system governs while another colors it.

CONTEST Two systems fight over the same structural role.

TRANSLATION One tradition performs the logic of another.

OCCUPATION Different systems inhabit the same measure without blending.

INFECTION One system gradually adopts another's behavior.

SOURCE SWAP Roles remain while instruments or traditions exchange places.

FALSE RECONCILIATION Systems align briefly, then separate.

MATERIAL TRANSFORMATION A physical sound source changes function without losing identity.

GRAMMAR TRANSFER The formal syntax of one music controls the timbres of another.

6.3 Reinvent the recombination process

After a method repeatedly produces recognizable results, mutate the method itself.

Examples:

- recombine relationships instead of genres
- combine failure modes
- combine transitions
- combine listener expectations
- combine performance ontologies
- combine incompatible systems while protecting one invariant

QUICK EXPERIMENT 5 — RECOMBINATION METHOD

Use the same two or three source traditions and generate:

A. ordinary blended hybrid B. alternating dominance C. structural occupation without stylistic reconciliation
D. grammar transfer

Compare whether the recombination strategy matters more than the ingredient list.

WRAP CONDITION

Genre invention becomes systematic without becoming predictable.

===== PHASE 7 — INTEGRATE THE LLM WORKSHOP
=====

PURPOSE

Define how the LLM collaborates without taking authorship away from Nathan's process.

LLM ROLES

ARCHIVIST Preserve exact inputs and outputs.

COMPILER Translate approved architecture into Suno fields.

CARTOGRAPHER Map voices, hooks, callbacks, triggers, and dependencies.

DEBUGGER Find broken wiring without erasing intentional conflict.

EXPERIMENT DESIGNER Create informative prompt variants.

LISTENER Describe the output that exists, not the prompt's intention.

THEORY KEEPER Update observations, hypotheses, rules, and counterexamples.

GENRELAB PARTNER Compile and recombine source material.

TASTE GUARD Flag when coherence has become conventionality or optimization has removed character.

RULE

The LLM proposes distinctions and architectures.

Nathan retains final authority over whether something is:

- a flaw
- productive friction
- an emergent gift
- conventional but useful
- merely not to his taste

WRAP CONDITION

The LLM can support both disciplined research and wild creative iteration without confusing the two.

===== PHASE 8 — BUILD A FIRST COMPLETE
WORKSHOP CYCLE =====

PURPOSE

Use one new song to test the complete system.

8.1 Select a manageable song problem

Prefer a song with:

- one strong invariant
- one unusual engine
- one clear counterforce
- a few exact lyric anchors
- enough room for local triggers
- no requirement to solve every prompting question at once

8.2 Build the Song Seed and Song Machine

8.3 Compile the first prompt

8.4 Generate the baseline pair

8.5 Audit both

8.6 Identify one control question

8.7 Run one fast paired experiment

8.8 Revise the prompt and the theory separately

8.9 Generate the next pair

8.10 Archive everything

WRAP CONDITION

We complete one song while also producing at least one useful model-behavior finding.

===== IMMEDIATE NEXT ACTIONS
=====

NEXT 1 Create the canonical project map and living-document structure.

NEXT 2 Extract the existing corpus into four first-pass documents:

- Working Theory
- Taste Profile

- Prompt Grammar
- Open Questions

NEXT 3 Choose three historical prompts for the Corpus Check: one wild success, one conventional success, one useful failure.

NEXT 4 Run the first active quick experiment:

LOCAL BINDING Distant definition versus definition at point of use.

NEXT 5 Use the results to revise the theory before designing the next experiment.

OPERATING CADENCE

Do not run an experiment simply because one is available.

Run one when:

- two hypotheses meaningfully disagree
- a proposed rule would substantially change prompt construction
- a compression decision risks destroying useful relationships
- v4.5 evidence may not transfer to v5.5
- a successful behavior needs replication
- a recurring failure needs isolation

FINAL PRINCIPLE

The workshop is not intended to eliminate uncertainty.

It is intended to:

- place uncertainty where it generates music
- reduce uncertainty where it only breaks communication
- preserve what Suno invents
- increase our ability to build complex song-worlds that remain coherent without becoming ordinary